EXPERIMENT 14: PACKET SNIFFING USING RAW SOCKETS

AIM

To write a program using raw sockets to capture and display network packet information.

THEORY

A packet sniffer captures packets arriving at a network interface and examines their headers. A raw socket can provide access to packet data at a lower network layer than a normal TCP or UDP socket. On Linux, packet capture generally requires elevated privileges.

ALGORITHM

1. Create a raw socket using the packet protocol.
2. Bind the socket to a network interface.
3. Receive packet bytes continuously.
4. Extract and display basic Ethernet header information such as destination MAC, source MAC, and EtherType.
5. Continue until the user stops the program.

PROGRAM (Python — Linux)

Save as `packet_sniffer.py`. The program displays Ethernet header details for packets captured on the selected interface.

```
import socket
import struct

interface = input("Interface (e.g. eth0, wlan0): ").strip()

# AF_PACKET captures link-layer frames on Linux.
sniffer = socket.socket(
    socket.AF_PACKET,
    socket.SOCK_RAW,
    socket.ntohs(0x0003)
)
sniffer.bind((interface, 0))

print("Sniffing packets on", interface, "(Ctrl+C to stop)")
try:
    while True:
        packet, address = sniffer.recvfrom(65535)
        if len(packet) < 14:
            continue

        dest_mac = ":".join(f"{b:02x}" for b in packet[0:6])
        src_mac = ":".join(f"{b:02x}" for b in packet[6:12])
        ethertype = struct.unpack("!H", packet[12:14])[0]

        print("\nDestination MAC:", dest_mac)
        print("Source MAC     :", src_mac)
        print("EtherType      : 0x%04x" % ethertype)
        print("Frame length   :", len(packet))
except KeyboardInterrupt:
    print("\nPacket capture stopped.")
finally:
    sniffer.close()
```

PROCEDURE

1. Save the program as `packet_sniffer.py`.
2. Open a Linux terminal in the file's directory.
3. Identify the interface using `ip link`.
4. Run the program with administrator privileges using `sudo python3 packet_sniffer.py` and enter the correct interface name.
5. Generate ordinary network traffic, observe the displayed packet headers, and press Ctrl+C to stop.

SAMPLE OUTPUT

```
Interface (e.g. eth0, wlan0): wlan0
Sniffing packets on wlan0 (Ctrl+C to stop)

Destination MAC: ff:ff:ff:ff:ff:ff
Source MAC     : 34:12:98:ab:cd:ef
EtherType      : 0x0806
Frame length   : 60

Packet capture stopped.
```

OBSERVATIONS

The program captures Ethernet frames and displays the source MAC address, destination MAC address, EtherType, and frame length. EtherType 0x0806 indicates ARP; 0x0800 indicates IPv4.

RESULT

A packet-sniffing program using a Linux raw socket was implemented to capture packets and display basic Ethernet header information.